

scopeblock — Project Plan

A plan to build the MSO44B oscilloscope automation library (Tektronix MSO44B + AFG31102, SCPI over VISA in Python), function by function. Each function is specified in three steps: **Analyze**, **Implement**, **Verify & Validate**.

How This Plan Is Organized

This plan lists the functions to build, in order — starting with a simple connection script, then each function of the MSO44B oscilloscope API. Every function is specified in the same three steps, so nothing is coded without first being specified and last being proven:

- **Analyze** — state precisely what the function does: its purpose, parameters (name, type, meaning), return value, side effects, error modes, and edge cases. This is the contract before any code.
- **Implement** — how to build it: the concrete SCPI sequence or algorithm, the module it lives in, and the coding practices that keep it correct, typed, and offline-testable.
- **Verify & Validate** — how we prove it: offline unit checks with a fake transport / synthetic waveforms (verification), live-bench checks (validation), and the explicit acceptance criterion.

The functions are ordered so each builds on the last: the connection script proves the physical link and becomes the VISA transport; connect / configure / acquire / read_waveform build the capture path; measure / compare / export / check_errors build the analysis and reporting path.

Function List

#	Function	Lives in	Key SCPI / basis
1	Initial script (bench_connect.py)	scripts/ (standalone)	open_resource, *IDN?, *CLS
2	connect() / identify()	transport/ + scope/mso44b.py	*IDN?
3	configure(config)	scope/mso44b.py	CH / HORizontal / TRIGger / DATA
4	acquire(timeout)	scope/mso44b.py	ACQUIRE:STATE / *OPC?
5	read_waveform(source)	scope/preamble.py	WFMOutpre? / CURVe?
6	measure(wf, kind)	analysis/measure.py	feature extraction
7	compare(wf, expected, tol)	analysis/compare.py	Result / Tolerance
8	export(wf, path, fmt)	io/export.py	CSV / npz / MF4
9	check_errors()	scope/mso44b.py	*ESR? / ALLEV?

The AFG31102 stimulus library and the config / reporting layers are later functions that reuse the same three-step template; this document details Functions 1-9.

Function 1: Initial Script — bench_connect.py

```
python scripts/bench_connect.py [--resource RES] [--timeout MS] -> exit code
```

The first script to be written, **scripts/bench_connect.py**: a small, standalone pyvisa program that opens a VISA session to the instrument, confirms its identity, and pushes a few commands. It is throwaway-grade scaffolding whose job is to de-risk the physical link (drivers, cabling, resource string) before any library code exists — and it becomes the blueprint for the VISA transport in Function 2.

ANALYZE — what it does

- **Purpose:** prove a live, two-way SCPI session to the MSO44B over VISA: open the resource, query *IDN?, quiet the protocol, read the error queue, and close cleanly.
- **In between:** create a ResourceManager, open the resource, set timeout and read/write terminations, write/query a handful of commands.
- **Error modes / edge cases:** SCOPE_RESOURCE unset (list visible resources and exit), instrument not found, VISA backend missing, query timeout (no reply).

Parameters

Parameter	Type / default	Meaning
--resource	str (default: \$SCOPE_RESOURCE)	VISA resource string, e.g. TCPIP0::192.168.0.10::INSTR or USB0::0x0699:::INSTR. Never hardcoded; the CLI flag overrides the environment variable.
--timeout	int ms (default: 5000)	Hard I/O cap so a no-reply raises instead of hanging.

Outputs

Output	Where	Meaning
Identity	stdout	The *IDN? string, e.g. TEKTRONIX,MSO44,,.
Error queue	stdout	The ALLEV? contents after hygiene — empty on a healthy session.
Exit code	process	0 = session opened, identified, closed clean; 2 = SCOPE_RESOURCE unset (resources listed); non-zero on open/timeout failure.

It returns no object — it is a script, not a library function; its result is the printed identity/error queue and the exit code.

IMPLEMENT — how to build it

- Use `pyvisa.ResourceManager()` and open with a context manager so the session always closes: `with rm.open_resource(res) as inst:`.
- Read the resource from the environment; if unset, print `rm.list_resources()` and exit with guidance — no hardcoded IP or serial.
- Set `inst.timeout` (a hard cap so a no-reply can never hang), and `read_termination / write_termination` to `"\n"`.
- Send session hygiene once: `HEADER OFF, VERBOSE OFF, *CLS` — so query replies are bare values and the error queue starts clean.
- **Best practice:** keep it under ~60 lines; guard the pyvisa import with a helpful install message; put everything under `if __name__ == "__main__":` so importing has no side effects; use `argparse` to override resource/timeout; do not enable any output or change instrument state beyond hygiene.

```
import os, sys, pyvisa
def main():
    res = os.environ.get("SCOPE_RESOURCE")
    rm = pyvisa.ResourceManager()
    if not res:
        print("SCOPE_RESOURCE unset. Visible:", rm.list_resources()); return 2
    with rm.open_resource(res) as inst:
```

```

    inst.timeout = 5000    # ms, hard cap
    inst.read_termination = inst.write_termination = "\n"
    print("IDN:", inst.query("*IDN?").strip())
    for c in ("HEAdEr OFF", "VERBoSe OFF", "*CLS"): inst.write(c)
    print("ERR:", inst.query("ALLEV?").strip())
    return 0
if __name__ == "__main__": sys.exit(main())

```

VERIFY & VALIDATE

- **Verify (offline):** run against the pyvisa-sim backend with a small YAML that defines a *IDN? reply, so the script's control flow (open, query, hygiene, close) is exercised with no hardware. Lint (ruff) and type-check (mypy) clean.
- **Validate (hardware):** run on the bench with the real SCOPE_RESOURCE; confirm *IDN? returns a Tektronix MSO44B identity, that ALLEV? reports no errors afterward, and that the session closes without a lingering lock.
- **Acceptance:** with the bench connected the script prints the correct identity and an empty error queue and exits 0; with SCOPE_RESOURCE unset it lists resources and exits non-zero without traceback.

Functions 2–9: MSO44B Oscilloscope Library

The oscilloscope library funnels everything through one contract, the **Waveform** (numpy t/v arrays in seconds/volts plus a metadata dict). All SCPI/VISA I/O sits behind a thin **Transport** interface (write / query / query_binary / close), so every function below is testable offline with a fake transport. The functions follow in build order.

Function 2: `connect()` / `identify()`

`VisaTransport(resource, timeout) ; MSO44B(transport).session_setup() -> str (the *IDN?)`

ANALYZE

Establishes a session and confirms instrument identity. In the library this is two pieces: a `VisaTransport` that opens the VISA resource and implements the Transport interface, and `MSO44B.session_setup()` that puts the instrument in a known, parse-friendly state and returns its identity.

Parameter	Type	Meaning
resource	str	VISA resource string (from <code>SCOPE_RESOURCE</code>), for the transport.
timeout	float (ms)	Hard I/O cap so a no-reply raises instead of hanging.
transport	Transport	The I/O seam MSO44B drives (VISA on the bench, fake in tests).

- **Returns:** the `*IDN?` string.
- **Side effects:** writes `HEADER OFF`, `VERBOSE OFF`, `*CLS`; marks the session ready.
- **Errors:** `TransportError` on open failure or timeout.

IMPLEMENT

- `VisaTransport` wraps `pyvisa`: `write/query` (strip whitespace) and `query_binary` (read the raw IEEE block via `read_raw`); translate `pyvisa` timeouts into `TransportError`.
- `session_setup()` queries `*IDN?`, sends the three hygiene commands, sets a `_session_ready` flag (so it runs once).
- **Best practice:** nothing outside `transport/` and `scope/` imports `pyvisa` or builds SCPI; the transport is a typed Protocol; import `pyvisa` lazily so the core package installs without it.

VERIFY & VALIDATE

- **Verify:** with a `FakeTransport` returning a canned `*IDN?`, assert `session_setup` returns the identity and issues `HEADER OFF` / `VERBOSE OFF` / `*CLS` in order; assert it is idempotent (a second call does not re-run hygiene).
- **Validate:** on the bench, assert the identity matches `/TEKTRONIX,MSO44/` and the error queue is empty.
- **Acceptance:** a real session opens, identifies, and reads back clean; the fake-transport unit test passes in CI with no hardware.

Function 3: `configure(config)`

`MSO44B(transport).configure(config) -> AppliedState`

ANALYZE

Pushes a complete scope setup — vertical, horizontal, trigger, and the waveform-transfer settings — and returns exactly what was sent, for traceability.

Parameter	Type	Meaning
config	ScopeSetup (or duck-typed)	channel, vertical_scale/offset, coupling, sample_rate/horizontal_scale, record_length, trigger_source/level/slope, encoding, byte_width.

- **Returns:** `AppliedState` (channel, the list of commands sent, the idn, a settings summary).
- **Side effects:** changes instrument state; calls `check_errors()` at the end.
- **Errors:** `ScopeError` if the instrument flags a command/execution error.

IMPLEMENT

- Read fields with `getattr(config, ..., default)` so any object exposing the attributes works; a local `send()` helper writes and appends to the command log.
- Emit in a deliberate order: `vertical (SELECT:CH ON, CH:SCALE, CH:OFFSET, CH:COUPLING) → horizontal (HORIZONTAL:SAMPLERATE / :SCALE, :RECORDLENGTH) → trigger (TRIGGER:A:TYPE EDGE, :EDGE:SOURCE, :LEVEL, :EDGE:SLOPE) → transfer (DATA:SOURCE/ENCODING/WIDTH/START/STOP)`.
- Only send optional settings when present (skip `None`), so a partial config leaves the rest at instrument defaults.
- **Best practice:** validate the config (pydantic) before touching hardware; record every command for the traceable `AppliedState`; do no scaling here.

VERIFY & VALIDATE

- **Verify:** drive configure with a fixed `ScopeSetup` against a `FakeTransport` that records writes; assert the exact SCPI strings (and their order) appear in `AppliedState.commands`; assert `None` fields emit nothing.
- **Validate:** after configure, query a couple of settings back (e.g. `HORIZONTAL:RECORDLENGTH?`) and assert they match; confirm `ALLEV?` is empty.
- **Acceptance:** a known setup produces the expected command log offline and reads back the same values on the bench with no error.

Function 4: `acquire(timeout, source)`

`MSO44B(transport).acquire(timeout=10.0, source="CH1") -> Waveform`

ANALYZE

Arms a single-sequence capture, blocks until it completes under a hard timeout, then reads one channel back as a scaled `Waveform`. A no-trigger is a typed error, never a hang.

Parameter	Type	Meaning
timeout	float (s)	Maximum wait for capture completion.
source	str	Channel to read back, e.g. "CH1".

- **Returns:** a `Waveform`.
- **Errors:** `AcquireTimeout` (a typed `ScopeError`) on no-trigger/timeout; `ScopeError` on an instrument error.

IMPLEMENT

- Ensure the session is set up; write `ACQUIRE:STOPAFTER SEQUENCE` then `ACQUIRE:STATE RUN` for a single sequence.
- Block on `*OPC?` inside `try/except TransportError` → raise `AcquireTimeout`; verify the reply is '1'; then `check_errors()` and return `read_waveform(source)`.

- **Best practice:** the transport enforces the wall-clock timeout, so the call is guaranteed to return or raise — it never blocks forever; keep the single-sequence semantics explicit rather than relying on free-run.

VERIFY & VALIDATE

- **Verify:** a FakeTransport whose *OPC? returns '1' and CURVe? returns synthetic bytes → assert a Waveform of the expected length; a fake that raises TransportError on *OPC? → assert AcquireTimeout; a fake returning a junk *OPC? → assert AcquireTimeout.
- **Validate:** with the AFG driving a signal, acquire returns a Waveform of record length; with no trigger, acquire raises AcquireTimeout within the timeout (measured with a stopwatch).
- **Acceptance:** good trigger yields a correct Waveform; no trigger fails fast and typed, within the configured timeout.

Function 5: read_waveform(source)

read preamble + CURVe? -> Waveform (scope/preamble.py owns the scaling)

ANALYZE

Pulls the transfer preamble and the raw sample block, then applies the affine transform to produce engineering units. This is the single place raw codes become volts and seconds.

- **Input:** the source channel; reads WFMOutpre? fields and the CURVe? IEEE block from the transport.
- **Returns:** a Waveform with t (s), v (V), dt, t0, and meta holding the raw preamble for provenance.
- **Conversion:** $v[n] = (\text{code}[n] - \text{YOFF}) \cdot \text{YMULT} + \text{YZERO}$; $t[n] = \text{XZERO} + (n - \text{PT_OFF}) \cdot \text{XINCR}$.
- **Errors:** ValueError on a malformed IEEE block or unsupported indefinite-length (#0) transfer.

IMPLEMENT

- Query preamble fields individually (WFMOutpre?:XINCR?, :YMULT?, ...) rather than positionally parsing WFMOutpre? — unambiguous across firmware.
- Pick the numpy dtype from the preamble: signed if BN_FMT starts 'RI', byte order '>' if BYT_OR is MSB, width from BYT_NR; strip the #<n><len> header, then np.frombuffer for a vectorized decode.
- Apply the affine transform vectorized; put the whole preamble in meta.
- **Best practice:** scaling lives only here; tolerate a trailing newline on the block; leave a TODO(confirm) to validate the PT_OFF sign against a real WFMOutpre? near the trigger.

VERIFY & VALIDATE

- **Verify:** synthesize integer codes + a known Preamble, run the transform, and assert volts/times match the formula exactly (round-trip); unit-test the IEEE parser (normal, trailing newline, #0 rejected) and dtype selection for signed/unsigned and both byte orders.
- **Validate:** capture a known AFG DC level and amplitude; assert decoded volts are within tolerance of commanded, and that len(v) equals NR_PT.
- **Acceptance:** the offline scaling round-trip is exact and the bench capture reads correct volts/time.

Function 6: measure(wf, kind, **params)

measure(wf, kind) -> float kinds: frequency, period, duty, vpp, vamlplitude, vhigh/vlow, mean, rms, rise_time, fall_time, edges

ANALYZE

Extracts one named feature from a Waveform. Measurement is robust feature extraction, not a sample-by-sample comparison.

Parameter	Type	Meaning
wf	Waveform	The scaled capture to measure.

Parameter	Type	Meaning
kind	str (case-insensitive)	Which feature: frequency, duty, vpp, ...
**params	kwargs	Per-measurement options (e.g. edge thresholds).

- **Returns:** a single float in the feature's units.
- **Errors:** `KeyError` for an unknown kind; `ValueError` when a signal lacks enough edges for a timing measurement.

IMPLEMENT

- Dispatch via a name→function map so kinds are data, not a long if/elif.
- Levels (V_{high} / V_{low} / V_{pp}): a histogram split about the median with a robust percentile fallback — **never** raw min/max (overshoot and single-sample spikes wreck those).
- Edges: a Schmitt (hysteresis) state so noise near a level can't double-count; crossing times linearly interpolated at the mid level for sub-sample resolution.
- Frequency from the mean of successive rising-edge periods; duty from high-time / period over complete cycles; rise/fall as the 10%-90% transition near the first edge.
- **Best practice:** pure functions on a `Waveform` (no I/O) so they are trivially unit-testable; return floats, not printed text.

VERIFY & VALIDATE

- **Verify:** synthetic generators with known freq/duty/levels; assert each measurement with `pytest.approx`; parametrize duty at 20/40/75%; add a noisy signal to prove edges don't double-count and a flat signal to prove amplitude 0 and a `ValueError` for timing.
- **Validate:** in the AFG loopback, measured frequency / amplitude / duty match the commanded values within tolerance.
- **Acceptance:** every measurement matches ground truth offline and the commanded values on the bench.

Function 7: `compare(wf, kind, expected, tol)`

`compare(wf, kind, expected, tol, requirement_id=None) -> Result`

ANALYZE

Measures a feature and renders a verdict against a tolerance band. A failure is a **Result** object carrying every value, not an exception — so a report can show exactly why something failed.

Parameter	Type	Meaning
expected	float	The commanded / specified value.
tol	Tolerance	abs and/or rel band (wider wins), or explicit min/max.
requirement_id	str None	Tag for ASPICE traceability (e.g. REQ-ECM-PWM-001).

- **Returns:** `Result` (name, measured, expected, lower, upper, passed, units, requirement_id) with `error` / `error_pct` properties and a readable `__str__`.

IMPLEMENT

- `Tolerance.band(expected)`: explicit min/max overrides everything; otherwise $half = \max(abs, rel \cdot |expected|)$ giving $[expected-half, expected+half]$ — the wider of absolute and relative wins.
- `compare` measures the kind, computes the band, and builds a `Result` with `passed = lower ≤ measured ≤ upper`; `compare_all` runs a list of Checks.
- **Best practice:** never raise on a failing check; keep all tolerance math in one place; always thread the `requirement_id` through for traceability.

VERIFY & VALIDATE

- **Verify:** a value inside the band → passed True, one outside → passed False, both returning a fully-populated Result; test abs-vs-rel wider-wins; test the explicit min/max override; test error_pct with expected 0 → nan.
- **Validate:** run a full TOML case (REQ-ECM-PWM-001) end to end and assert all checks pass on a good signal and the right check fails when the stimulus is deliberately detuned.
- **Acceptance:** pass and fail paths both yield correct, reportable Results, offline and on the bench.

Function 8: export(wf, path, fmt)

export(wf, path, fmt=None) -> None (load(path) -> Waveform reloads it)

ANALYZE

Persists a Waveform so captures are traceable and re-loadable. File load must emit the exact same Waveform shape as live capture, so nothing downstream can tell them apart.

Parameter	Type	Meaning
path	str Path	Destination file.
fmt	str None	csv / npz / mf4; defaults to the file extension.

- **Returns:** None (writes a file).
- **Formats:** csv (time,volts + a commented meta header), npz (lossless arrays + meta, used by load), mf4 (automotive, optional via asammdf).
- **Errors:** ValueError on an unsupported format; a clear RuntimeError if MF4 is requested without asammdf installed.

IMPLEMENT

- Dispatch on fmt (or the suffix); csv writes a JSON meta comment then rows at %.12g; npz uses np . savez with meta as a JSON string; mf4 lazy-imports asammdf and raises a helpful message if missing.
- load_waveform reverses each: npz is lossless; csv recovers arrays and meta from the header.
- **Best practice:** keep meta JSON-serializable; the optional MF4 dependency stays behind a lazy import; guarantee the round-trip shape equals live capture.

VERIFY & VALIDATE

- **Verify:** export to npz then load and assert arrays + meta are equal (lossless round-trip); csv round-trip equal within float precision; unsupported fmt → ValueError; MF4 test uses `pytest.importorskip("asammdf")`.
- **Validate:** none required — export is pure and fully covered offline.
- **Acceptance:** a captured Waveform survives export/reload unchanged in every supported format available.

Function 9: check_errors()

MS044B(transport).check_errors() -> None (raises ScopeError on instrument error)

ANALYZE

Turns instrument-reported errors into exceptions rather than silent failures. Called at sync points (after configure and after acquire).

- **Input:** none (reads the instrument status).
- **Returns:** None on a clean status.
- **Errors:** raises ScopeError with the ALLEV? text when the event-status register flags a command (CME), execution (EXE), device (DDE) or query (QYE) error.

IMPLEMENT

- Read *ESR?; if it ANDs with the error mask (bits 5/4/3/2, i.e. 0b0011_1100), query ALLEV? for the human-readable events and raise `ScopeError(f"...ESR={esr}: {events}")`.
- **Best practice:** fail loud, never swallow; mask only the error bits so benign status bits don't trip it; rely on the session's *CLS to start each run with a clean queue.

VERIFY & VALIDATE

- **Verify:** a FakeTransport that returns an ESR with an error bit set plus canned ALLEV? text → assert `ScopeError` with that text; a clean ESR (0) → no raise.
- **Validate:** send a deliberately malformed command on the bench and assert `check_errors` raises; confirm the next run starts clean after *CLS.
- **Acceptance:** real and simulated instrument errors surface as typed exceptions; a healthy instrument passes silently.

Each function is complete when its Verify (offline) and Validate (bench) acceptance criteria pass. Later functions — the AFG31102 stimulus library, TOML config, JUnit/Robot reporting — reuse this same Analyze / Implement / Verify & Validate template. · scopeblock, KPIT Internship — V&V Automation.